Seq graph

Retiming starts from a sequential circuit graph , not just a Boolean network

The idea

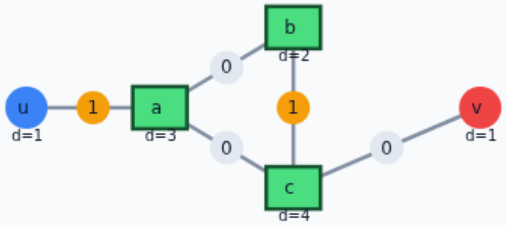
- Nodes carry combinational delay; edges carry register counts w
- Our toy has five nodes and five edges

Topology

SEQUENTIAL GRAPH

Topology

Step 1 / 5 · topo · module01-01-sequential-graph



Explanation

Five nodes: PI u, comb a/b/c, PO v, plus shortcut a → c.

- DAG

```
nodes=5
```

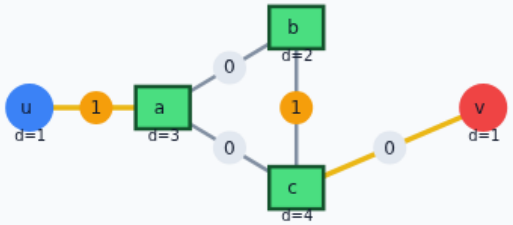
Topology

PI/PO

SEQUENTIAL GRAPH

PI/PO

Step 2 / 5 · pi · module01-01-sequential-graph



Explanation

Blue circle u is the primary input; red v is the primary output.

- Ports

```
u / v
```

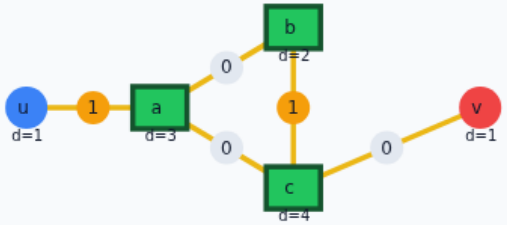
PI/PO

Comb boxes

SEQUENTIAL GRAPH

Comb boxes

Step 3 / 5 · comb · module01-01-sequential-graph



Explanation

Green boxes a, b, c carry combinational delay.

- delay d

```
a=3 b=2 c=4
```

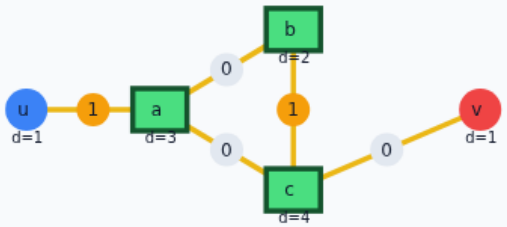
Comb boxes

Five edges

SEQUENTIAL GRAPH

Five edges

Step 4 / 5 · edges · module01-01-sequential-graph



Explanation

Edge labels show register weight w. Five edges total.

- w labels

```
edges=5
```

Five edges

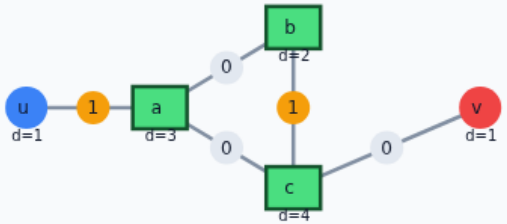
Stats

SEQUENTIAL GRAPH

Stats

Step 5 / 5 · stats · module01-01-sequential-graph

{"nodes":5,"edges":5,"registers":2,"co



Explanation

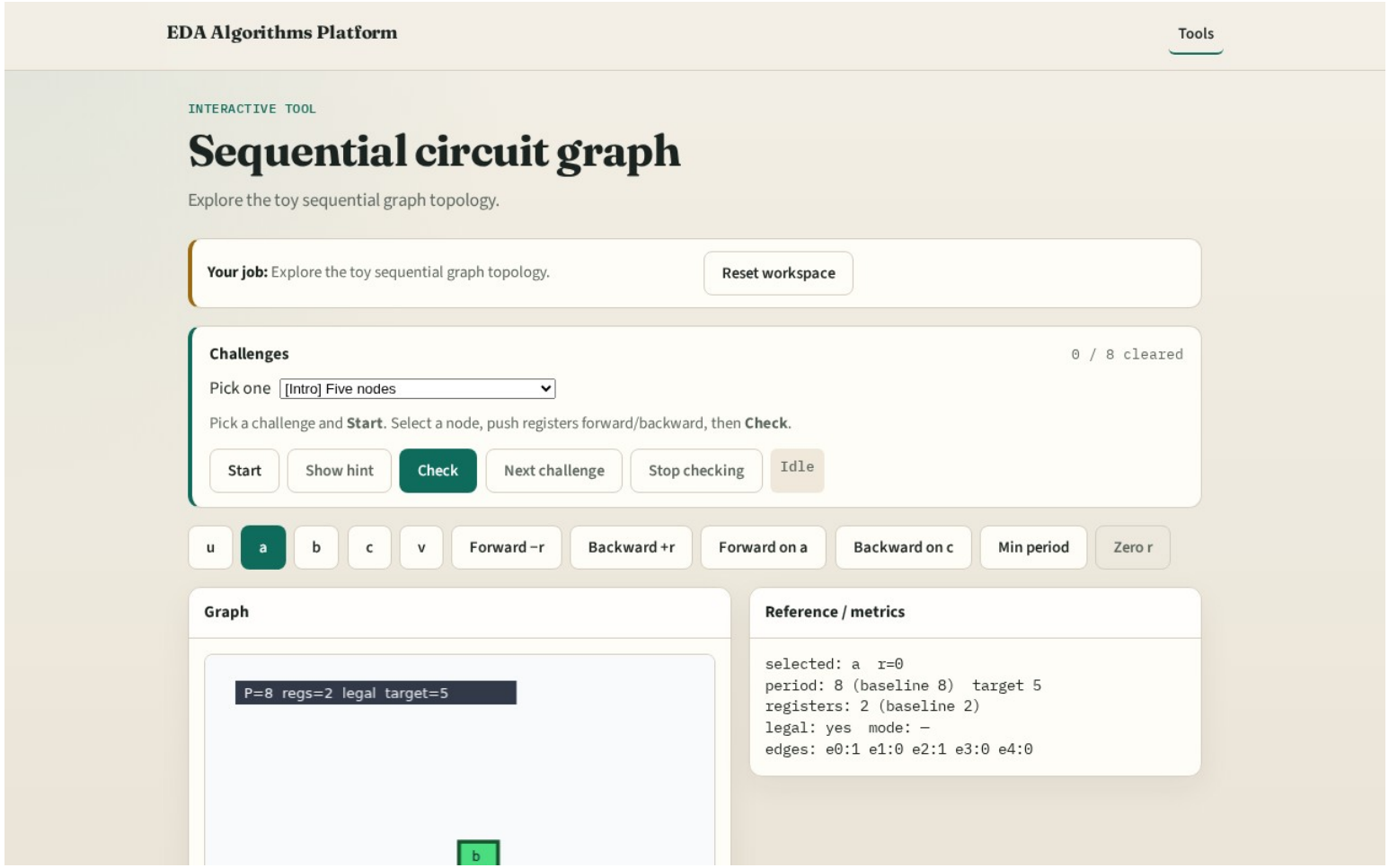
GOLDENS: 5 nodes, 5 edges, 3 comb, 2 base registers.

- graphStats

```
regs=2
```

Stats

Browser lab track



Browser lab starter

Implement track

- Load tiny_retime.json; call graph_stats , expect 5 nodes, 5 edges

Pitfalls

- Treating edge w as delay
- Confusing PO v with a comb node

Your turn

- Next: edge register counts in detail